

Table of Contents

1. Problem Statement.....	4
2. Summary of Data Analysis.....	4
3. MapReduce Analysis	5
3.1 Task 1: Transaction Count by Class	5
<i>Java Code</i>	5
<i>Execution</i>	6
<i>Result Summary</i>	6
3.2 Task 2: Total Amount by Class.....	7
<i>Java Code</i>	7
<i>Execution</i>	8
<i>Result Summary</i>	8
3.3 Task 3: Average Amount by Class.....	9
<i>Java Code</i>	9
<i>Execution</i>	10
<i>Result Summary</i>	10
3.4 Task 4: Max Amount by Class.....	11
<i>Java Code</i>	11
<i>Execution</i>	12
<i>Result Summary</i>	12
3.5 Task 5: Amount Range Distribution	13
<i>Java Code</i>	13
<i>Execution</i>	14
<i>Result Summary</i>	14
4. Pig Analysis.....	15
4.1 Task 1: Filter Fraud Transactions.....	15
<i>Pig Script</i>	15
<i>Execution</i>	16
<i>Observation</i>	17
4.2 Task 2: Filter High-Value Transactions	19
<i>Pig Script</i>	19
<i>Execution</i>	20
<i>Observation</i>	23
4.3 Task 3: Transaction Count by Class	23
<i>Pig Script</i>	23
<i>Execution</i>	23
<i>Observation</i>	23

4.4 Task 4: Amount Statistics by Class	24
<i>Pig Script</i>	24
<i>Execution</i>	24
<i>Observation</i>	24
4.5 Task 5: Top 20 Highest-Amount Transactions	25
<i>Pig Script</i>	25
<i>Execution</i>	25
<i>Observation</i>	25
4.6 Task 6: Join with Class Labels.....	26
<i>Pig Script</i>	26
<i>Execution</i>	26
<i>Observation</i>	28
4.7 Task 7: Amount Range Distribution	29
<i>Pig Script</i>	29
<i>Execution</i>	29
<i>Observation</i>	29
4.8 Task 8: Overall Fraud Rate Percentage.....	30
<i>Pig Script</i>	30
<i>Execution</i>	30
<i>Observation</i>	30
5. Conclusion	31

1. Problem Statement

Credit card fraud is one of the most significant challenges facing today's digital payment ecosystem. Millions of transactions occur every day, and even though only a tiny fraction of them are fraudulent, they cause massive financial losses for institutions and customers alike. Analyzing such a huge volume of transactional data quickly and efficiently is difficult with traditional systems.

The goal of this project is to analyze a real-world Credit Card Fraud Detection dataset (568,630 transactions, 31 columns) using Big Data technologies — Hadoop MapReduce and Apache Pig — in order to:

- Identify statistical differences between Fraud and Legitimate transactions
- Understand the distribution and pattern of transaction Amounts
- Demonstrate the effectiveness of distributed processing on a large-scale dataset
- Compare the two approaches — MapReduce (low-level, Java-based) and Pig (high-level, script-based)

2. Summary of Data Analysis

Dataset : <https://www.kaggle.com/datasets/nelgiriyeewithana/credit-card-fraud-detection-dataset-2023>

Attribute	Details
Total Transactions (Rows)	568,630
Total Columns	31 (id, V1–V28, Amount, Class)
File Size	≈ 324 MB
Class 0 (Legitimate)	284,315 (50%)
Class 1 (Fraud)	284,315 (50%)
V1 – V28	Features anonymized via PCA (for confidentiality)
Amount	Monetary value of the transaction

The dataset is notably and perfectly balanced — both the Fraud and Legitimate classes contain exactly the same number of transactions (284,315 each). This makes it an ideal balanced dataset for Machine Learning model training, free of class bias. Columns V1 through V28 have been transformed using Principal Component Analysis (PCA) to protect sensitive transaction details (such as merchant and location), so the Amount and Class columns form the primary basis of this analysis.

3. MapReduce Analysis

The five MapReduce tasks are presented in detail below — including the purpose, the complete Java code (Mapper, Reducer, and Driver classes), and the actual execution output for each.

3.1 Task 1: Transaction Count by Class

This task groups the entire dataset by the Class column (0 = Legitimate, 1 = Fraud) and counts the total number of transactions in each group. The Mapper reads the Class column of each record and emits it as the key; the Reducer sums all values for each key to compute the total count.

Java Code

```
// Task 1: Find the total Transaction Count for each Class (0 = Legitimate, 1 = Fraud)
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

import java.io.IOException;

public class Task1_TransactionCountByClass {

    public static class CountMapper extends Mapper<Object, Text, Text, IntWritable> {

        private final static IntWritable ONE = new IntWritable(1);
        private Text classKey = new Text();

        @Override
        protected void map(Object key, Text value, Context context)
            throws IOException, InterruptedException {

            String line = value.toString();
            if (line.startsWith("id,") return; // skip header

            String[] fields = line.split(",");
            if (fields.length < 31) return;

            String cls = fields[30].trim(); // Class column (0 or 1)
            classKey.set(cls.equals("1") ? "Fraud" : "Legitimate");
            context.write(classKey, ONE);
        }
    }

    public static class CountReducer extends Reducer<Text, IntWritable, Text, IntWritable> {

        @Override
        protected void reduce(Text key, Iterable<IntWritable> values, Context context)
            throws IOException, InterruptedException {

            int sum = 0;
            for (IntWritable val : values) {
                sum += val.get();
            }
            context.write(key, new IntWritable(sum));
        }
    }
}
```

```

public static void main(String[] args) throws Exception {
    if (args.length != 2) {
        System.err.println("Usage: Task1_TransactionCountByClass <input path> <output path>");
        System.exit(-1);
    }

    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "Transaction Count By Class");

    job.setJarByClass(Task1_TransactionCountByClass.class);
    job.setMapperClass(CountMapper.class);
    job.setCombinerClass(CountReducer.class);
    job.setReducerClass(CountReducer.class);

    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);

    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));

    System.exit(job.waitForCompletion(true) ? 0 : 1);
}
}

```

Execution

```

C:\fraud_project\project2>hdfs dfs -cat /fraud_project/output/task1/part-r-00000
Fraud      284315
Legitimate  284315

```

Result Summary

Class	Total Transactions
Fraud	284,315
Legitimate	284,315

The result shows the dataset is perfectly balanced — both the Fraud and Legitimate classes contain an equal number of transactions (284,315 each).

3.2 Task 2: Total Amount by Class

This task groups records by Class and sums the Amount of all transactions in each group to compute the total. The Mapper emits the Class and Amount of each record; the Reducer sums those Amount values.

Java Code

```
// Task 2: Find the total Transaction Amount for each Class
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.DoubleWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

import java.io.IOException;

public class Task2_TotalAmountByClass {

    public static class SumMapper extends Mapper<Object, Text, Text, DoubleWritable> {

        private Text classKey = new Text();
        private DoubleWritable amount = new DoubleWritable();

        @Override
        protected void map(Object key, Text value, Context context)
            throws IOException, InterruptedException {

            String line = value.toString();
            if (line.startsWith("id,") ) return;

            String[] fields = line.split(",");
            if (fields.length < 31) return;

            try {
                String cls = fields[30].trim();
                double amt = Double.parseDouble(fields[29]); // Amount column

                classKey.set(cls.equals("1") ? "Fraud" : "Legitimate");
                amount.set(amt);
                context.write(classKey, amount);
            } catch (NumberFormatException e) {
                // skip malformed data
            }
        }
    }

    public static class SumReducer extends Reducer<Text, DoubleWritable, Text, DoubleWritable> {

        @Override
        protected void reduce(Text key, Iterable<DoubleWritable> values, Context context)
            throws IOException, InterruptedException {

            double sum = 0.0;
            for (DoubleWritable val : values) {
                sum += val.get();
            }
            context.write(key, new DoubleWritable(sum));
        }
    }

    public static void main(String[] args) throws Exception {
        if (args.length != 2) {
            System.err.println("Usage: Task2_TotalAmountByClass <input path> <output path>");
        }
    }
}
```

```

        System.exit(-1);
    }

    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "Total Amount By Class");

    job.setJarByClass(Task2_TotalAmountByClass.class);
    job.setMapperClass(SumMapper.class);
    job.setCombinerClass(SumReducer.class);
    job.setReducerClass(SumReducer.class);

    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(DoubleWritable.class);

    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));

    System.exit(job.waitForCompletion(true) ? 0 : 1);
}
}

```

Execution

```

C:\fraud_project\project2>hdfs dfs -cat /fraud_project/output/task2/part-r-00000
Fraud      3.428157045350006E9
Legitimate  3.419261324399996E9

```

Result Summary

Class	Total Amount
Fraud	3,428,157,045.35
Legitimate	3,419,261,324.40

Although the total Amount of Fraud transactions is slightly higher, the totals for both classes are very close — another reflection of the balanced nature of the dataset.

3.3 Task 3: Average Amount by Class

This task computes the average Transaction Amount for each Class. The Mapper emits the Class and Amount of each record; the Reducer tracks both the Sum and the Count and applies $\text{Average} = \text{Sum} / \text{Count}$.

Java Code

```
// Task 3: Find the average Transaction Amount for each Class
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

import java.io.IOException;

public class Task3_AverageAmountByClass {

    // Mapper key = Class, value = Amount (as Text; parsed in the Reducer)
    public static class AvgMapper extends Mapper<Object, Text, Text, Text> {

        private Text classKey = new Text();
        private Text amountVal = new Text();

        @Override
        protected void map(Object key, Text value, Context context)
            throws IOException, InterruptedException {

            String line = value.toString();
            if (line.startsWith("id,")) return;

            String[] fields = line.split(",");
            if (fields.length < 31) return;

            String cls = fields[30].trim();
            classKey.set(cls.equals("1") ? "Fraud" : "Legitimate");
            amountVal.set(fields[29]);
            context.write(classKey, amountVal);
        }
    }

    // Reducer keeps sum and count to compute the average
    public static class AvgReducer extends Reducer<Text, Text, Text, Text> {

        @Override
        protected void reduce(Text key, Iterable<Text> values, Context context)
            throws IOException, InterruptedException {

            double sum = 0;
            long count = 0;

            for (Text val : values) {
                try {
                    sum += Double.parseDouble(val.toString());
                    count++;
                } catch (NumberFormatException e) {
                    // skip bad record
                }
            }

            double avg = (count == 0) ? 0 : sum / count;
            context.write(key, new Text(String.format("%.2f", avg)));
        }
    }
}
```

```

public static void main(String[] args) throws Exception {
    if (args.length != 2) {
        System.err.println("Usage: Task3_AverageAmountByClass <input path> <output path>");
        System.exit(-1);
    }

    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "Average Amount By Class");

    job.setJarByClass(Task3_AverageAmountByClass.class);
    job.setMapperClass(AvgMapper.class);
    job.setReducerClass(AvgReducer.class);

    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(Text.class);

    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));

    System.exit(job.waitForCompletion(true) ? 0 : 1);
}

```

Execution

```

C:\fraud_project\project2>hdfs dfs -cat /fraud_project/output/task3/part-r-00000
Fraud      12057.60
Legitimate  12026.31

```

Result Summary

Class	Average Amount
Fraud	12,057.60
Legitimate	12,026.31

The average Amount is nearly identical for both classes — meaning average Amount alone is not sufficient to distinguish a Fraud transaction from a Legitimate one, which highlights why the other features (V1–V28) are needed.

3.4 Task 4: Max Amount by Class

This task finds the maximum Transaction Amount within each Class. The Mapper emits the Class and Amount; the Reducer finds the maximum value among all values for each key.

Java Code

```
// Task 4: Find the maximum Transaction Amount for each Class
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.DoubleWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

import java.io.IOException;

public class Task4_MaxAmountByClass {

    public static class MaxMapper extends Mapper<Object, Text, Text, DoubleWritable> {

        private Text classKey = new Text();
        private DoubleWritable amount = new DoubleWritable();

        @Override
        protected void map(Object key, Text value, Context context)
            throws IOException, InterruptedException {

            String line = value.toString();
            if (line.startsWith("id,") ) return;

            String[] fields = line.split(",");
            if (fields.length < 31) return;

            try {
                String cls = fields[30].trim();
                double amt = Double.parseDouble(fields[29]);

                classKey.set(cls.equals("1") ? "Fraud" : "Legitimate");
                amount.set(amt);
                context.write(classKey, amount);
            } catch (NumberFormatException e) {
                // skip
            }
        }
    }

    public static class MaxReducer extends Reducer<Text, DoubleWritable, Text, DoubleWritable> {

        @Override
        protected void reduce(Text key, Iterable<DoubleWritable> values, Context context)
            throws IOException, InterruptedException {

            double max = Double.MIN_VALUE;
            for (DoubleWritable val : values) {
                if (val.get() > max) {
                    max = val.get();
                }
            }
            context.write(key, new DoubleWritable(max));
        }
    }
}
```

```

public static void main(String[] args) throws Exception {
    if (args.length != 2) {
        System.err.println("Usage: Task4_MaxAmountByClass <input path> <output path>");
        System.exit(-1);
    }

    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "Max Amount By Class");

    job.setJarByClass(Task4_MaxAmountByClass.class);
    job.setMapperClass(MaxMapper.class);
    job.setCombinerClass(MaxReducer.class);
    job.setReducerClass(MaxReducer.class);

    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(DoubleWritable.class);

    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));

    System.exit(job.waitForCompletion(true) ? 0 : 1);
}
}

```

Execution

```

C:\fraud_project\project2>hdfs dfs -cat /fraud_project/output/task4/part-r-00000
Fraud      24039.93
Legitimate  24039.93

```

Result Summary

Class	Max Amount
Fraud	24,039.93
Legitimate	24,039.93

Interestingly, the maximum Amount is exactly identical for both classes (24,039.93) — further evidence that this dataset was synthetically balanced.

3.5 Task 5: Amount Range Distribution

This task divides each transaction's Amount into four ranges/buckets (0-100, 100-1000, 1000-10000, 10000+) and counts how many transactions fall into each range. The Mapper classifies the Amount into the correct range using if-else logic and emits it as the key; the Reducer counts the occurrences.

Java Code

```
// Task 5: Divide Amount into several ranges/buckets and count how many Transactions fall in each
range
// Range: 0-100, 100-1000, 1000-10000, 10000+
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

import java.io.IOException;

public class Task5_AmountRangeDistribution {

    public static class RangeMapper extends Mapper<Object, Text, Text, IntWritable> {

        private final static IntWritable ONE = new IntWritable(1);
        private Text rangeKey = new Text();

        @Override
        protected void map(Object key, Text value, Context context)
            throws IOException, InterruptedException {

            String line = value.toString();
            if (line.startsWith("id,") return;

            String[] fields = line.split(",");
            if (fields.length < 31) return;

            try {
                double amt = Double.parseDouble(fields[29]);
                String range;

                if (amt < 100) {
                    range = "0-100";
                } else if (amt < 1000) {
                    range = "100-1000";
                } else if (amt < 10000) {
                    range = "1000-10000";
                } else {
                    range = "10000+";
                }

                rangeKey.set(range);
                context.write(rangeKey, ONE);
            } catch (NumberFormatException e) {
                // skip
            }
        }
    }

    public static class RangeReducer extends Reducer<Text, IntWritable, Text, IntWritable> {

        @Override
        protected void reduce(Text key, Iterable<IntWritable> values, Context context)
```

```

        throws IOException, InterruptedException {

            int sum = 0;
            for (IntWritable val : values) {
                sum += val.get();
            }
            context.write(key, new IntWritable(sum));
        }
    }

    public static void main(String[] args) throws Exception {
        if (args.length != 2) {
            System.err.println("Usage: Task5_AmountRangeDistribution <input path> <output path>");
            System.exit(-1);
        }

        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "Amount Range Distribution");

        job.setJarByClass(Task5_AmountRangeDistribution.class);
        job.setMapperClass(RangeMapper.class);
        job.setCombinerClass(RangeReducer.class);
        job.setReducerClass(RangeReducer.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}

```

Execution

```

C:\fraud_project\project2>hdfs dfs -cat /fraud_project/output/task5/part-r-00000
0-100    1190
100-1000    21320
1000-10000    213580
10000+    332540

```

Result Summary

Amount Range	Transaction Count
0 - 100	1,190
100 - 1,000	21,320
1,000 - 10,000	213,580
10,000+	332,540

The largest share of transactions falls in the 10,000+ range (58.5%), indicating that this dataset is dominated by relatively high-value transactions.

4. Pig Analysis

The eight Apache Pig tasks are presented below — including the purpose, the complete Pig Latin script, and the actual execution output for each. Notably, Pig's high-level dataflow syntax accomplishes the same work in far less code than MapReduce.

4.1 Task 1: Filter Fraud Transactions

All transactions with Class = 1 (Fraud) are filtered out. Data is read with LOAD, and the FILTER operator keeps only the Fraud records.

Pig Script

```
-- Task 1: Filter and display only Fraud (Class = 1) transactions

transactions = LOAD 'creditcard_2023.csv' USING PigStorage(',')
    AS (id:int, V1:double, V2:double, V3:double, V4:double, V5:double, V6:double,
        V7:double, V8:double, V9:double, V10:double, V11:double, V12:double,
        V13:double, V14:double, V15:double, V16:double, V17:double, V18:double,
        V19:double, V20:double, V21:double, V22:double, V23:double, V24:double,
        V25:double, V26:double, V27:double, V28:double, Amount:double, Class:int);

-- discard the header line
txn_clean = FILTER transactions BY id IS NOT NULL;

-- filter only Fraud transactions
fraud_txns = FILTER txn_clean BY Class == 1;

fraud_result = FOREACH fraud_txns GENERATE id, Amount, Class;

STORE fraud_result INTO 'output/task1_fraud_transactions' USING PigStorage(',');
-- DUMP fraud_result;
```

Execution

```
C:\fraud_project\project2\pig>type output\task1_fraud_transactions\part-m-00000
541,1534.53,1
623,10554.68,1
4920,2785.98,1
6108,15897.18,1
6329,3492.59,1
6331,3875.51,1
6334,4271.95,1
6336,13210.79,1
6338,20487.28,1
6427,3996.01,1
6446,12162.55,1
6472,9890.39,1
6529,10676.68,1
6609,3732.02,1
6641,5774.97,1
6717,12864.25,1
6719,15293.66,1
6734,10289.34,1
6774,3883.37,1
6820,12955.72,1
6870,7853.77,1
6882,11468.71,1
6899,1415.95,1
6903,1944.95,1
6971,11144.83,1
8296,2240.3,1
8312,20543.56,1
8335,19991.71,1
8615,8608.29,1
8617,15418.96,1
8842,9325.59,1
8845,3442.52,1
8972,16591.45,1
9035,17575.44,1
9179,19107.17,1
9252,20540.23,1
9487,23690.3,1
9509,17207.31,1
10204,22669.63,1
10484,2026.44,1
10497,10166.27,1
10498,9823.26,1
10568,14115.67,1
10630,11318.75,1
10690,20738.5,1
```

10801,20845.88,1
10891,12309.86,1
10897,13273.34,1
11343,10199.35,1
11710,22521.64,1
11841,21937.62,1
11880,8971.3,1
12070,8784.92,1
12108,18516.7,1
12261,8135.36,1
12369,22917.97,1
14104,3638.56,1
14170,6775.71,1
14197,13729.65,1
14211,11281.61,1
14338,3524.26,1
15166,6730.66,1
15204,10267.44,1
15225,15888.21,1
15451,18514.59,1
15476,5744.14,1
15506,5305.39,1
15539,22520.54,1
15566,14397.5,1
15736,11452.46,1
15751,11725.7,1
15781,20257.4,1
15810,4055.92,1
16415,5048.69,1
16780,17124.97,1
16863,18882.39,1
17317,925.61,1
17366,8102.86,1
17407,19918.66,1
17453,1111.14,1
17480,19813.32,1
18466,19836.5,1
18472,9102.6,1
18773,22107.91,1
18809,6431.21,1
20198,2507.41,1
23308,17483.72,1
23422,1011.31,1
26802,12223.99,1
27362,9968.96,1
27627,18480.29,1
27738,9108.17,1

```
42635,20897.34,1
42674,14622.93,1
42696,16177.13,1
42700,1415.24,1
42741,914.28,1
42756,18021.51,1
42769,8868.2,1
42784,6744.04,1
42856,13483.83,1
42887,4018.9,1
42936,19822.93,1
42945,13381.71,1
42958,18980.93,1
43061,14555.88,1
43160,9054.73,1
43204,15937.43,1
43428,23733.54,1
43624,11472.94,1
43681,14073.2,1
43773,15933.46,1
44001,5930.29,1
44091,2014.78,1
44223,274.39,1
44270,755.28,1
44556,123.01,1
45203,16926.7,1
45732,18842.57,1
46909,10327.59,1
46918,22741.29,1
46998,6481.77,1
47802,72.82,1
48094,6720.0,1
50211,14096.05,1
50537,22758.16,1
52466,11818.26,1
52521,1474.4,1
52584,13423.56,1
53591,2441.81,1
53794,19481.75,1
55401,11664.38,1
56703,7366.07,1
57248,6727.68,1
57470,12647.0,1
57615,20231.06,1
58422,5718.24,1
58761,7680.29,1
```

Observation

A total of 284,315 Fraud transactions were found — this exactly matches the result of MapReduce Task 1, confirming that both tools produce the same correct result.

4.2 Task 2: Filter High-Value Transactions

High-value transactions with Amount > 10000 are filtered out — this helps identify suspiciously large transactions.

Pig Script

```
-- Task 2: Filter High-Value transactions where Amount > 10000

transactions = LOAD 'creditcard_2023.csv' USING PigStorage(',')
    AS (id:int, V1:double, V2:double, V3:double, V4:double, V5:double, V6:double,
        V7:double, V8:double, V9:double, V10:double, V11:double, V12:double,
        V13:double, V14:double, V15:double, V16:double, V17:double, V18:double,
        V19:double, V20:double, V21:double, V22:double, V23:double, V24:double,
        V25:double, V26:double, V27:double, V28:double, Amount:double, Class:int);

txn_clean = FILTER transactions BY id IS NOT NULL;

high_value = FILTER txn_clean BY Amount > 10000;

high_value_result = FOREACH high_value GENERATE id, Amount, Class;

STORE high_value_result INTO 'output/task2_high_value_transactions' USING PigStorage(',');
-- DUMP high_value_result;
```

Execution

```
C:\fraud_project\project2\pig>type output\task2_high_value_transactions\part-m-00000
43595,21923.05,0
43596,19433.33,0
43599,12797.4,0
43600,17923.33,0
43601,13442.56,0
43602,10434.71,0
43603,18811.28,0
43604,13956.44,0
43605,20986.5,0
43606,12932.42,0
43607,12044.37,0
43608,18518.82,0
43609,15489.17,0
43610,19112.41,0
43612,22581.99,0
43613,10130.0,0
43616,21468.2,0
43617,21404.32,0
43619,21514.81,0
43620,19593.24,0
43623,14743.42,0
43624,11472.94,1
43626,21092.89,0
43627,14431.06,0
43630,23839.25,0
43631,22760.06,0
43632,18207.84,0
43635,17729.12,0
43639,22640.1,0
43640,23628.7,0
43643,15964.62,0
43647,21088.94,0
43648,19240.58,0
43649,10548.98,0
43650,10271.45,0
43651,19341.52,0
43656,24001.55,0
43657,10336.24,0
43658,10666.86,0
43663,20323.99,0
43664,10786.28,0
43666,15828.95,0
43667,20673.43,0
43668,12735.74,0
43674,10014.64,0
43676,18725.06,0
```

44536,12806.27,0
44540,22926.33,0
44542,13565.79,0
44543,11112.32,0
44544,16187.51,0
44545,10825.72,0
44547,17024.52,0
44548,14105.67,0
44549,21584.05,0
44550,16113.19,0
44551,15770.21,0
44553,10317.81,0
44554,16137.71,0
44555,10750.08,0
44557,18487.96,0
44558,14453.4,0
44559,20016.68,0
44560,11330.81,0
44561,11945.67,0
44562,22612.93,0
44563,24029.0,0
44564,16953.92,0
44566,15148.24,0
44567,15246.96,0
44569,17548.35,0
44571,23450.55,0
44573,17095.97,0
44574,20561.48,0
44575,19574.25,0
44577,18196.34,0
44579,20492.96,0
44580,18374.67,0
44581,15268.95,0
44582,18180.88,0
44583,10294.23,0
44584,12575.25,0
44585,13878.46,0
44587,16561.14,0
44589,11042.75,0
44593,14713.9,0
44594,13266.58,0
44595,20494.16,0
44596,14636.86,0
44598,22618.64,0
44599,17986.32,0
44600,22569.0,0
44602,23931.24,0

```
48755,11056.5,0
48757,21047.02,0
48760,15433.97,0
48761,17657.84,0
48762,10215.15,0
48763,13371.23,0
48766,15796.23,0
48767,15398.11,0
48770,13299.25,0
48771,20657.97,0
48772,20207.14,0
48773,19679.28,0
48775,20681.75,0
48776,16981.19,0
48778,15358.06,0
48780,22903.0,0
48781,23018.36,0
48787,17964.33,0
48788,16202.23,0
48789,16376.18,0
48790,12834.97,0
48793,10957.21,0
48794,20918.47,0
48796,16061.25,0
48800,19235.78,0
48804,13366.32,0
48805,18802.08,0
48807,10397.76,0
48809,20596.17,0
48812,11209.34,0
48813,12919.41,0
48814,14889.1,0
48816,13878.8,0
48817,15622.06,0
48819,21690.59,0
48820,12585.07,0
48823,14863.56,0
48824,20527.63,0
48827,22402.02,0
48828,13507.4,0
48829,23696.58,0
48830,10139.48,0
48831,10034.0,0
48833,11847.06,0
48835,10368.31,0
48836,19199.99,0
48839,13656.92,0
```

Observation

A total of 332,540 transactions have an Amount greater than 10,000 — about 58.5% of the total data, consistent with the 10000+ range result from MapReduce Task 5.

4.3 Task 3: Transaction Count by Class

Records are grouped by Class using the GROUP operator, and the COUNT() function is used to find the total number of records in each group.

Pig Script

```
-- Task 3: Find the total Transaction Count for each Class

transactions = LOAD 'creditcard_2023.csv' USING PigStorage(',')
    AS (id:int, V1:double, V2:double, V3:double, V4:double, V5:double, V6:double,
        V7:double, V8:double, V9:double, V10:double, V11:double, V12:double,
        V13:double, V14:double, V15:double, V16:double, V17:double, V18:double,
        V19:double, V20:double, V21:double, V22:double, V23:double, V24:double,
        V25:double, V26:double, V27:double, V28:double, Amount:double, Class:int);

txn_clean = FILTER transactions BY id IS NOT NULL;

grouped_by_class = GROUP txn_clean BY Class;

class_count = FOREACH grouped_by_class GENERATE
    group AS Class,
    COUNT(txn_clean) AS TotalTransactions;

STORE class_count INTO 'output/task3_count_by_class' USING PigStorage(',');
-- DUMP class_count;
```

Execution

```
C:\fraud_project\project2\pig>type output\task3_count_by_class\part-r-00000
0,284315
1,284315
```

Observation

This fully matches the result of MapReduce Task 1 (Class 0 = Legitimate = 284,315; Class 1 = Fraud = 284,315).

4.4 Task 4: Amount Statistics by Class

Using SUM(), AVG(), MAX(), and MIN() functions in a single FOREACH statement, the complete Amount statistics for each Class are computed together.

Pig Script

```
-- Task 4: Compute Total, Average, Max, and Min Amount together for each Class

transactions = LOAD 'creditcard_2023.csv' USING PigStorage(',')
AS (id:int, V1:double, V2:double, V3:double, V4:double, V5:double, V6:double,
    V7:double, V8:double, V9:double, V10:double, V11:double, V12:double,
    V13:double, V14:double, V15:double, V16:double, V17:double, V18:double,
    V19:double, V20:double, V21:double, V22:double, V23:double, V24:double,
    V25:double, V26:double, V27:double, V28:double, Amount:double, Class:int);

txn_clean = FILTER transactions BY id IS NOT NULL;

grouped_by_class = GROUP txn_clean BY Class;

amount_stats = FOREACH grouped_by_class GENERATE
    group AS Class,
    ROUND(SUM(txn_clean.Amount)) AS TotalAmount,
    ROUND(AVG(txn_clean.Amount)) AS AvgAmount,
    MAX(txn_clean.Amount) AS MaxAmount,
    MIN(txn_clean.Amount) AS MinAmount;

STORE amount_stats INTO 'output/task4_amount_stats_by_class' USING PigStorage(',');
-- DUMP amount_stats;
```

Execution

```
C:\fraud_project\project2\pig>type output\task4_amount_stats_by_class\part-r-00000
0,3419261324,12026,24039.93,50.12
1,3428157045,12058,24039.93,50.01
```

Observation

Column order: Class, TotalAmount, AvgAmount, MaxAmount, MinAmount. This single Pig script accomplishes what MapReduce Tasks 2, 3, and 4 do combined — a good example of Pig's conciseness.

4.5 Task 5: Top 20 Highest-Amount Transactions

Records are sorted in descending order of Amount using ORDER BY, and LIMIT extracts the top 20 transactions with the highest Amount.

Pig Script

```
-- Task 5: Find the Top 20 transactions with the highest Amount (descending order)

transactions = LOAD 'creditcard_2023.csv' USING PigStorage(',')
    AS (id:int, V1:double, V2:double, V3:double, V4:double, V5:double, V6:double,
        V7:double, V8:double, V9:double, V10:double, V11:double, V12:double,
        V13:double, V14:double, V15:double, V16:double, V17:double, V18:double,
        V19:double, V20:double, V21:double, V22:double, V23:double, V24:double,
        V25:double, V26:double, V27:double, V28:double, Amount:double, Class:int);

txn_clean = FILTER transactions BY id IS NOT NULL;

txn_selected = FOREACH txn_clean GENERATE id, Amount, Class;

-- sort in descending order by Amount
txn_sorted = ORDER txn_selected BY Amount DESC;

-- take only the first 20 records
top20_txns = LIMIT txn_sorted 20;

STORE top20_txns INTO 'output/task5_top20_transactions' USING PigStorage(',');
-- DUMP top20_txns;
```

Execution

```
C:\fraud_project\project2\pig>type output\task5_top20_transactions\part-r-00000
473871,24039.93,1
85369,24039.93,0
15509,24039.88,0
182117,24039.85,0
376626,24039.82,1
515776,24039.76,1
436922,24039.61,1
137650,24039.55,0
32125,24039.55,0
241276,24039.53,0
352843,24039.5,1
481625,24039.48,1
108338,24039.48,0
306134,24039.47,1
63586,24039.46,0
296087,24039.46,1
178769,24039.45,0
69064,24039.43,0
34041,24039.43,0
491753,24039.39,1
```

Observation

The Top 20 includes both Fraud and Legitimate transactions — proving that a high Amount alone does not indicate fraud.

4.6 Task 6: Join with Class Labels

The main transactions data is JOINed with a small lookup dataset `class_labels.csv` (Class → Label, RiskLevel) to attach a human-readable Label and RiskLevel to every transaction.

Pig Script

```
-- Task 6: JOIN transactions with class_labels.csv to display a readable label (Legitimate/Fraud, RiskLevel)

transactions = LOAD 'creditcard_2023.csv' USING PigStorage(',')
AS (id:int, V1:double, V2:double, V3:double, V4:double, V5:double, V6:double,
    V7:double, V8:double, V9:double, V10:double, V11:double, V12:double,
    V13:double, V14:double, V15:double, V16:double, V17:double, V18:double,
    V19:double, V20:double, V21:double, V22:double, V23:double, V24:double,
    V25:double, V26:double, V27:double, V28:double, Amount:double, Class:int);

txn_clean = FILTER transactions BY id IS NOT NULL;
txn_selected = FOREACH txn_clean GENERATE id, Amount, Class;

class_labels = LOAD 'class_labels.csv' USING PigStorage(',')
AS (Class:int, Label:chararray, RiskLevel:chararray);

class_labels_clean = FILTER class_labels BY Class IS NOT NULL;

-- JOIN based on the Class column
joined_data = JOIN txn_selected BY Class, class_labels_clean BY Class;

joined_result = FOREACH joined_data GENERATE
    txn_selected::id AS id,
    txn_selected::Amount AS Amount,
    class_labels_clean::Label AS Label,
    class_labels_clean::RiskLevel AS RiskLevel;

-- show a small portion of the result (first 50 records instead of all 500K+)
joined_sample = LIMIT joined_result 50;

STORE joined_sample INTO 'output/task6_joined_with_labels' USING PigStorage(',');
-- DUMP joined_sample;
```

Execution

```
C:\fraud_project\project2\pig>type output\task6_joined_with_labels\part-r-00000
58837,9831.69,Legitimate,Low
58838,6761.8,Legitimate,Low
58839,563.15,Legitimate,Low
58840,21180.21,Legitimate,Low
58841,1441.21,Legitimate,Low
58842,11000.75,Legitimate,Low
58843,22982.32,Legitimate,Low
58844,787.52,Legitimate,Low
58845,11122.74,Legitimate,Low
58846,9945.36,Legitimate,Low
58847,3478.83,Legitimate,Low
58848,16687.45,Legitimate,Low
58849,13326.33,Legitimate,Low
58850,6975.23,Legitimate,Low
58851,10515.44,Legitimate,Low
58852,22071.32,Legitimate,Low
58853,17443.43,Legitimate,Low
58854,4850.29,Legitimate,Low
58855,4846.36,Legitimate,Low
58856,9398.67,Legitimate,Low
58857,19894.75,Legitimate,Low
58858,11004.38,Legitimate,Low
58859,9861.98,Legitimate,Low
58860,5385.53,Legitimate,Low
58861,13024.88,Legitimate,Low
58862,15428.13,Legitimate,Low
58863,3030.05,Legitimate,Low
58864,10910.95,Legitimate,Low
58865,8447.42,Legitimate,Low
58866,11059.35,Legitimate,Low
58867,10502.39,Legitimate,Low
58868,3193.6,Legitimate,Low
58869,2932.21,Legitimate,Low
58870,6500.67,Legitimate,Low
58871,15110.05,Legitimate,Low
58872,20057.53,Legitimate,Low
58873,22120.22,Legitimate,Low
58874,6551.64,Legitimate,Low
58875,17648.36,Legitimate,Low
58876,21574.57,Legitimate,Low
58877,9967.16,Legitimate,Low
58878,18189.77,Legitimate,Low
58879,16365.35,Legitimate,Low
58880,23550.3,Legitimate,Low
58881,9835.21,Legitimate,Low
```

```
58857,19894.75,Legitimate,Low
58858,11004.38,Legitimate,Low
58859,9861.98,Legitimate,Low
58860,5385.53,Legitimate,Low
58861,13024.88,Legitimate,Low
58862,15428.13,Legitimate,Low
58863,3030.05,Legitimate,Low
58864,10910.95,Legitimate,Low
58865,8447.42,Legitimate,Low
58866,11059.35,Legitimate,Low
58867,10502.39,Legitimate,Low
58868,3193.6,Legitimate,Low
58869,2932.21,Legitimate,Low
58870,6500.67,Legitimate,Low
58871,15110.05,Legitimate,Low
58872,20057.53,Legitimate,Low
58873,22120.22,Legitimate,Low
58874,6551.64,Legitimate,Low
58875,17648.36,Legitimate,Low
58876,21574.57,Legitimate,Low
58877,9967.16,Legitimate,Low
58878,18189.77,Legitimate,Low
58879,16365.35,Legitimate,Low
58880,23550.3,Legitimate,Low
58881,9835.21,Legitimate,Low
58882,4932.73,Legitimate,Low
58883,20946.51,Legitimate,Low
58884,2917.84,Legitimate,Low
58885,23930.54,Legitimate,Low
58886,9964.64,Legitimate,Low
```

Observation

Two different data sources are combined using the JOIN operator — an important Pig capability that converts a raw numeric Class value into a readable label.

4.7 Task 7: Amount Range Distribution

A new AmountRange column is created for every record using a nested bincond (ternary condition), then GROUP and COUNT are used to find how many transactions fall in each range, sorted in descending order with ORDER BY.

Pig Script

```
-- Task 7: Divide Amount into ranges/buckets and count how many Transactions fall in each range

transactions = LOAD 'creditcard_2023.csv' USING PigStorage(',')
AS (id:int, V1:double, V2:double, V3:double, V4:double, V5:double, V6:double,
    V7:double, V8:double, V9:double, V10:double, V11:double, V12:double,
    V13:double, V14:double, V15:double, V16:double, V17:double, V18:double,
    V19:double, V20:double, V21:double, V22:double, V23:double, V24:double,
    V25:double, V26:double, V27:double, V28:double, Amount:double, Class:int);

txn_clean = FILTER transactions BY id IS NOT NULL;

-- use a nested bincond (ternary condition) to add an AmountRange column to every record
txn_with_range = FOREACH txn_clean GENERATE
    id, Amount, Class,
    (Amount < 100 ? '0-100' :
     (Amount < 1000 ? '100-1000' :
      (Amount < 10000 ? '1000-10000' : '10000+'))) AS AmountRange;

grouped_by_range = GROUP txn_with_range BY AmountRange;

range_distribution = FOREACH grouped_by_range GENERATE
    group AS AmountRange,
    COUNT(txn_with_range) AS TotalTransactions;

range_distribution_sorted = ORDER range_distribution BY TotalTransactions DESC;

STORE range_distribution_sorted INTO 'output/task7_amount_range_distribution' USING PigStorage(',');
-- DUMP range_distribution_sorted;
```

Execution

```
C:\fraud_project\project2\pig>type output\task7_amount_range_distribution\part-r-00000
10000+,332540
1000-10000,213580
100-1000,21320
0-100,1190
```

4.8 Task 8: Overall Fraud Rate Percentage

GROUP ALL is used to separately compute the total transaction count and the Fraud transaction count, then the CROSS operator combines these two single-record relations to calculate the overall Fraud Rate percentage.

Pig Script

```
-- Task 8: Compute the overall Fraud Rate (%) = (Total Fraud Transactions / Total Transactions) * 100

transactions = LOAD 'creditcard_2023.csv' USING PigStorage(',')
AS (id:int, V1:double, V2:double, V3:double, V4:double, V5:double, V6:double,
    V7:double, V8:double, V9:double, V10:double, V11:double, V12:double,
    V13:double, V14:double, V15:double, V16:double, V17:double, V18:double,
    V19:double, V20:double, V21:double, V22:double, V23:double, V24:double,
    V25:double, V26:double, V27:double, V28:double, Amount:double, Class:int);

txn_clean = FILTER transactions BY id IS NOT NULL;

-- find the total number of Transactions
grouped_all = GROUP txn_clean ALL;
total_count = FOREACH grouped_all GENERATE COUNT(txn_clean) AS TotalTxns;

-- filter only Fraud Transactions and find their count
fraud_txns = FILTER txn_clean BY Class == 1;
grouped_fraud = GROUP fraud_txns ALL;
fraud_count = FOREACH grouped_fraud GENERATE COUNT(fraud_txns) AS FraudTxns;

-- CROSS the two small single-record relations together, then compute the Percentage
combined = CROSS total_count, fraud_count;

fraud_rate = FOREACH combined GENERATE
    total_count::TotalTxns AS TotalTransactions,
    fraud_count::FraudTxns AS FraudTransactions,
    ROUND((double)fraud_count::FraudTxns / (double)total_count::TotalTxns * 100) AS
    FraudRatePercentage;

STORE fraud_rate INTO 'output/task8_fraud_rate' USING PigStorage(',');
-- DUMP fraud_rate;
```

Execution

```
C:\fraud_project\project2\pig>type output\task8_fraud_rate\part-r-00000
568630,284315,50
```

Observation

Column order: TotalTransactions, FraudTransactions, FraudRatePercentage. The result shows that exactly 50% (284,315) of the 568,630 total transactions are Fraud — confirming the dataset was deliberately balanced for Machine Learning training.

5. Conclusion

In this project, a total of five MapReduce tasks and eight Pig tasks were successfully implemented and executed on the Credit Card Fraud Detection Dataset. The key observations from the analysis are:

- The dataset is perfectly balanced — 50% Fraud and 50% Legitimate transactions.
- Average and maximum Amount alone are not sufficient to distinguish Fraud from Legitimate transactions — these values are nearly identical across both classes.
- About 58.5% of transactions have an Amount greater than 10,000, indicating a predominance of relatively high-value transactions.
- Both MapReduce and Pig produced identical results, confirming the correctness of the analysis.
- Using Pig's high-level syntax, the same analysis could be completed with significantly less code than MapReduce, especially when combining multiple aggregate functions (SUM, AVG, MAX, MIN) and performing JOIN operations.

Overall, this project demonstrates that both Hadoop MapReduce and Apache Pig are effective tools for analyzing large-scale transactional data, and that with proper practical application they can play a significant role in solving real-world problems such as fraud detection.